

## Chapter 3 — Summary

**Goal:** Understands backward proof concepts and tactics.

### 1. Core Concepts

#### 1. Tactics

##### Key

Tactics operate on the goal, which consists of the proposition  $Q$  that we want to prove and of a local context  $C$ .

##### Memo

intro,  $\forall$  is a function (implication),

```
theorem fst_of_two_props :  
  ∀ a b : Prop, a → b → a := by  
  intro a b  
  intro ha hb  
  apply ha
```

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b → c) : a → c := by  
  intro ha  
  apply hbc  
  apply hab  
  apply ha
```

#### 2. Basic Tactics

##### Key

intro, apply, exact, assumption, rfl, ac\_rfl

##### Memo

- apply works for the target of the goal if it is not specified.
- conversions:  $\alpha, \beta, \delta, \zeta, \eta, \iota$ 
  - These are the computations for definitional equality, and they are used to simplify expressions in Lean.
- ac\_rfl is similar to rfl, but it can handle equalities that involve commutative and associative operations, such as addition and multiplication.

#### 3. Metavariable and Unification

##### Key

Unification is the process to find substitutions for metavariables that make two expressions equal. In Lean, unification is used to match the goal with the conclusion of a theorem or lemma, and to find the appropriate substitutions for the metavariables in the theorem or lemma.

Metavariables are placeholders for terms that are not yet known, and unification is the process of finding a substitution for metavariables that makes two expressions equal.

##### Memo

It relies on term rewriting, symbolic computation, and type inference.

```
theorem And_swap (a b : Prop) : a ∧ b → b ∧ a := by  
  intro hab  
  apply And.intro  
  apply And.right  
  exact hab  
  apply And.left  
  exact hab
```

#### 4. Introduction and Elimination Rules

### Key

An introduction rule for a logical symbol (e.g.,  $\wedge$ ) is a theorem whose conclusion has that symbol as the outermost symbol. Dually, an elimination rule has the symbol in an assumption. For each logical symbol, the introduction rules tell us how to prove a proposition with that symbol as the outermost position. By contrast, the elimination rules tell us how we must have proved such a proposition.

### Memo

- Natural deduction is a proof system that uses introduction and elimination rules to derive conclusions from premises. Lean is based on natural deduction, and its tactics are designed to help users apply these rules effectively.
- Or.inl, Or.inr, Or.elim,
- And.intro, And.left, And.right,
- Not.intro, Not.elim,
- Imp.intro, Imp.elim,
- Iff.intro, Iff.elim
- True.intro, False.elim,
- Not is defined as  $\neg P := P \rightarrow \text{False}$ .

## 5. for all is a function (implication)

### Key

In Lean, the universal quantifier  $\forall$  is treated as a function type, which means that proving a statement of the form  $\forall x : A, P x$  is equivalent to proving that for any  $x$  of type  $A$ , the proposition  $P x$  holds. This is closely related to implication, as it allows us to assume an arbitrary element and prove a property about it.

$\forall x, P x = x \rightarrow P x$

```
theorem f5_if (h :  $\forall n : \mathbb{N}, f\ n = n$ ) :  $f\ 5 = 5$  := by
  exact h 5
```

## 6. Classical logic

### Key

- Classical.em :  $\forall a : \text{Prop}, a \vee \neg a$
- Classical.byContradiction :  $(\neg ?a \rightarrow \text{False}) \rightarrow ?a$

## 7. Equality

### Key

- Eq.refl :  $\forall a, a = a$
- Eq.symm :  $?a = ?b \rightarrow ?b = ?a$
- Eq.trans :  $?a = ?b \rightarrow ?b = ?c \rightarrow ?a = ?c$
- Eq.subst :  $?a = ?b \rightarrow ?P\ ?a \rightarrow ?P\ ?b$

### Memo

- Look at the code following and recall how it works. Look closely at the metavariables and subgoals.
- apply is the core of backward proof.

```
theorem Eq_trans_symm { $\alpha : \text{Type}$ } (a b c :  $\alpha$ ) (hab :  $a = b$ ) (hcb :  $c = b$ ) :  $a = c$  := by
  apply Eq.trans
  { exact hab }
  { apply Eq.symm
    exact hcb }
```

## 8. Rewriting

### Key

Rewriting is the process of replacing a term in an expression with another term that is equal to it, according to some equality or equivalence relation. In Lean, rewriting can be done using the `rw` tactic, which allows us to apply equalities to transform expressions in our goals or hypotheses.

### Memo

- `rw` tactic can be used to rewrite using equalities in the goal or in the hypotheses.
- The `rw` tactic can also be used with the `←` symbol to rewrite in the opposite direction.
- `simp` is `rw` with a set of simplification rules, which can be used to simplify expressions in the goal or in the hypotheses.

## 9. Mathematical Induction

### Key

Recursive inductive types can be defined in Lean, and mathematical induction is a proof technique that allows us to prove properties of these types by showing that they hold for the base case and that if they hold for an arbitrary case, they also hold for the next case. In Lean, we can use the `induction` tactic to perform mathematical induction on inductive types.

### Memo

```
theorem add_succ (m n : N) : add (Nat.succ m) n = Nat.succ (add m n) := by
induction n with
| zero => rfl
| succ n' ih => simp [add, ih]
```

```
theorem add_comm (m n : N) : add m n = add n m := by
induction n with
| zero => simp [add, add_zero]
| succ n' ih => simp [add, add_succ, ih]
```

```
theorem add_assoc (l m n : N) : add (add l m) n = add l (add m n) := by
induction n with
| zero => rfl
| succ n' ih => simp [add, ih]
```

## 2. Ask → Answer

### AA

What is the core of the backward proof? How does it relate to the application of tactics and the use of metavariables? How does unification work in Lean, and how does it relate to metavariables and the application of tactics?

### AA

Write down introduction and elimination rules for each logical symbol. How do these rules relate to the application of tactics in Lean? How does the use of introduction and elimination rules affect the structure of proofs in Lean?

### AA

How does the universal quantifier  $\forall$  relate to implication in Lean? How does this relationship affect the way we prove statements involving  $\forall$  in Lean? How does the use of  $\forall$  as a function type affect the structure of proofs in Lean?

### AA

Explain `intro`, `apply`, `exact`, `assumption`, `rfl`, `ac_rfl` tactics in Lean.

## AA

Explain `rw`, `simp` tactics in Lean. What are the conversions `alpha`, `beta`, `delta`, `zeta`, `eta`, `iota`? How do they relate to the simplification of expressions in Lean?

### 3. Remaining Questions

- How does unification work in Lean, and how does it relate to metavariables and the application of tactics? Matching is one form of unification, but there are more general forms of unification that can handle more complex cases.
- When is proper to use classical logic without construction of terms? How does it relate to the law of excluded middle and proof by contradiction? What are the implications of using classical logic in Lean, and how does it affect the constructiveness of proofs?

